

Theoretical–Simulation Bridge Project

Formation-First VSIM · Identity Entropy Outputs · Scale-Residual Tracking

Status	Internal theoretical planning document
Audience	VSEPR-SIM / VSIM developers
Purpose	Bridge between theoretical identity / scale notation and simulation outputs
Target	v5.2.0-internal

WARNING This is not a public-facing claims document. It is a design bridge. No need to dress it up in fake certainty like a grant proposal written by a committee of sleep-deprived raccoons.

Contents

§1 Project Intent	2
§2 Core Principle	2
§3 Theoretical Basis	2
§4 Formation as Required Simulation Context	3
§5 Identity-Entropy Output Layer	4
§6 Proposed Variables	4
§7 Scale Coalescence and Residual Tracking	5
§8 Formation Memory	6
§9 Script Integration	7
§10C++ Implementation Skeleton	8

§1 Project Intent

This chapter formalizes the transition from VSIM as a simulation scripting language into VSIM as a formation-history and identity-information language. The immediate goal is to connect theoretical identity notation, scale coalescence, residual structure, and recoverable information to concrete simulation artifacts. The system should not merely output positions, velocities, energies, and rendered views. It should also output what the simulation can infer about how a material formed, what information survived projection across scales, what residual structure was hidden during packing, and how much identity content remains recoverable after formation, interaction, or coalescence.

The bridge must remain disciplined: theoretical identity variables belong in derived output sidecars, overlays, reports, and analysis layers — not in mutable ground-truth state. VSIM already separates runtime intent, output records, render state, and validation behavior; render state is explicitly not supposed to mutate truth state, which gives this project the right architectural boundary instead of letting theory seep into the kernel like coolant into a bad weld.

§2 Core Principle

The core rule is:

Every material simulation must preserve formation context, and every identity-information output must remain derived from state history rather than written back into state.

Formation describes how the object entered its starting condition. Identity-entropy outputs describe what is lost, preserved, hidden, or recoverable as the simulation evolves. These outputs are not replacements for physics. They are an interpretive bridge between the raw simulation state and the theoretical identity framework.

This follows the existing VSIM direction where parsed sections are transformed into runtime objects through an intent bridge, including material, environment, and run configuration. Formation handling already exists as a planned or partially wired runtime concept through formation stages and field-ramp evaluation, with broader runtime execution deferred toward v5.2.0.

§3 Theoretical Basis

The theoretical side of the bridge comes from the identity-matrix framework. The identity reference defines the universal object in its compact form:

UNIVERSAL OBJECT — COMPACT FORM

$$P_i = (I_i, H_i, \Omega_w)$$

where I_i is the identity matrix, H_i is the deterministic hash matrix, and Ω_w is the world seed.

The extended form adds residual structure and recoverable information:

UNIVERSAL OBJECT — EXTENDED FORM

$$P_i = (I_i, H_i, R_i, \tilde{\zeta}_i^*, J_i)$$

R_i = residual structure · $\tilde{\zeta}_i^*$ = true identity-location · J_i = recoverable information / distinguishability

The same reference defines the deterministic path: identity, hash, world seed, and evolution rules together define the reproducible trajectory.

DETERMINISTIC PATH

$$\gamma_i(t) = F(I_i, H_i, \Omega_w, t)$$

For the simulation bridge, the most important theoretical object is J_i , because it provides a scalar measure of how much lower-scale structure survives projection into a higher-scale representation. The notation reference explicitly distinguishes J_i (an information/recovery measure belonging to an object) from Q_n (the count of active identities at scale n , a simulation inventory count). That distinction matters: one describes information content, the other describes population.

§ 4 Formation as Required Simulation Context

Formation data becomes mandatory because a material state without formation context is under-described. A gas mixture, cast bracket, powder bed, welded pipe, crystal surface, or bead aggregate does not begin as an abstract coordinate list. It has a preparation path. That preparation path affects density, defects, residual stress, grain state, packing, surface roughness, thermal state, and transport behavior.

Minimum formation fields should include:

VSIM Script -- Formation Block (minimum)

```
[formation]
enabled      = true
method       = "declared"
history      = "tracked"
formation_time = 0.0

[formation.initial_state]
temperature  = 300.0
pressure     = 101325.0
phase        = "solid"
density      = 1.0
defect_state = "unknown"
```

```
grain_state      = "unknown"
```

This block is not decoration. It is the upstream condition for material interpretation. The simulation can still run without perfect formation knowledge, but the validation layer should mark missing formation as `incomplete`, especially for material, gas, pipe, surface, powder, and structural simulations.

§5 Identity-Entropy Output Layer

The proposed new output layer is a compact derived sidecar with extension `.ie` (Identity Entropy). This file carries identity-information variables per frame and per identity-bearing object. It links to `.dynx`, manifests, reports, and viewer overlays, but must not mutate particle state.

```
[outputs.identity_entropy]
enabled          = true
path             = "output/run.ie"
format           = "table"
precision        = 6
attach_to_viewer = true
write_per_frame  = true
write_summary    = true
variables = [
  "J",
  "dataloss",
  "dataentropy",
  "residual",
  "hidden_activity",
  "projection_loss",
  "recoverability",
  "coalescence_loss",
  "lineage_depth",
  "state_quota",
  "entropy_flux",
  "formation_memory"
]
```

This layer belongs beside analysis outputs, symbolic traces, event logs, dashboards, and manifest records. Existing VSIM output infrastructure already supports many distinct artifact types — analysis JSON, event JSON, symbolic traces, reports, dashboards, manifests, and audit records — so this is an extension of the output doctrine rather than a random new species of file goblin.

§6 Proposed Variables

The first official variable set should include at least twelve fields.

Variable	Meaning	Maps to
J	Recoverable information / distinguishability	J_i
<code>dataloss</code>	Direct identity information loss	$1 - J_i$
<code>dataentropy</code>	Entropy-like spread/ambiguity of identity channels	derived
<code>residual</code>	Magnitude of packing or projection residual	$ R_i $
<code>hidden_activity</code>	Activity in channels not visible in projected state	derived
<code>projection_loss</code>	Loss from mapping full identity-space into observable	derived
<code>recoverability</code>	Inverse measure of residual/projection loss	$1 - \text{proj_loss}$
<code>coalescence_loss</code>	Loss from new higher-scale identity formation	$R_{\text{coal}}^{(n)}$
<code>lineage_depth</code>	Formation/coalescence layers behind an object	derived
<code>state_quota</code>	Count of active identities at a scale	Q_n
<code>entropy_flux</code>	Time derivative of identity entropy	$\dot{S}_{\text{id}}$
<code>formation_memor</code>	Fraction of formation history recoverable	M_f

A compact row format:

```

frame id J   dataloss dataentropy residual hidden_act proj_loss recov coal_loss
lin state_q entr_flux form_mem flags
0  17  0.9321  0.0679  0.1142  0.0831  0.2200  0.0714  0.9286  0.0000  2
1  0.0042  0.7810  0
1  17  0.9260  0.0740  0.1219  0.0887  0.2351  0.0782  0.9218  0.0025  2
1  0.0049  0.7744  0

```

Human-readable enough to debug. Cryptic enough to look like it came out of a basement at Los Alamos. Balance, finally.

§7 Scale Coalescence and Residual Tracking

The identity reference defines scale coalescence as the formation of a new higher-scale identity when lower-scale identities satisfy a bond/interactability condition and a collective binding score. The coalescence gate is:

COALESCENCE GATE

$$G_{\text{coal}}(n) = \mathbf{1}[m \geq 3] \cdot \mathbf{1}[\bar{d}^{(n)} \leq \ell_{\text{bond}}^{(n)}] \cdot \mathbf{1}[B_n(C_n) \geq \tau_B]$$

m = member count · $\bar{d}^{(n)}$ = mean inter-member distance · $\ell_{\text{bond}}^{(n)}$ = bond length threshold

$$B_n(C_n) = \text{collective binding score} \quad \cdot \quad \tau_B = \text{binding threshold}$$

When the gate fires, a new identity is projected at the next scale:

NEW HIGHER-SCALE IDENTITY

$$I_{(n+1)}^* = \Pi_n(C_n)$$

The coalescence residual captures internal structure hidden by the new identity formation:

COALESCENCE RESIDUAL

$$R_{\text{coal}}^{(n)} = C_n - A_n(I_{(n+1)}^*)$$

$R_{\text{coal}}^{(n)}$ = structure hidden by new identity formation. Whatever the higher-scale projection cannot represent is residual.

For simulation purposes this becomes an output rule: when a formation, clustering, bonding, packing, or phase event creates a higher-scale object, emit `coalescence_loss` and `residual` fields into the identity-entropy sidecar. This should not assert that the simulation has “discovered” a real new ontology. Yes, science requires this annoying humility.

§8 Formation Memory

Formation memory is the bridge between formation-first scripting and identity-entropy output.

FORMATION MEMORY

$$M_f = \frac{\text{recoverable formation descriptors}}{\text{total declared formation descriptors}}$$

Formation descriptors may include:

temperature history	pressure history	phase path
cooling rate	deposition order	packing sequence
grain state	defect seed	residual stress
surface finish	mixing pathway	

A high M_f value means the current state still carries measurable evidence of its formation history. A low value means the formation path has been erased, thermalized, randomized, projected away, or become hidden in residual structure.

This is especially important for:

cast parts	welded joints	powder beds
gas mixtures	pipe interiors	surface residence simulations
crystal defects	irradiated materials	

The obvious engineering value is that two materials with identical composition may behave differently because they formed differently. Humanity did need several thousand years of metallurgy to rediscover “history matters,” but here we are.

§9 Script Integration

A minimal VSIM script exercising the full bridge. The older `[export]` output system is kept intact; the `[outputs.identity_entropy]` layer is additive. The doctrine matters more than the bracket costume.

VSIM Script -- identity_entropy_surface_demo · v5.2.0-internal

```
[project]
name      = "identity_entropy_surface_demo"
version   = "v5.2.0-internal"
seed_base = 72001
determinism = true

[formation]
enabled      = true
method       = "deposition"
history      = "tracked"
formation_time = 0.0

[formation.initial_state]
temperature  = 300.0
pressure     = 101325.0
phase        = "solid"
defect_state = "tracked"
surface_state = "roughened"

[material]
formula      = "C"
structure    = "graphite"
phase        = "solid"

[system]
mode         = "surface_interaction"
space        = "3d"

[[surface]]
type          = "graphite_layer"
roughness     = 0.05
warm_minimum_beads = 100

[particles.projectile]
count        = 128
type         = "generic_bead"
velocity     = [0.0, 0.0, -1.5]
height       = 15.0
```

```

[run]
mode          = "md"
max_steps     = 8000
dt_fs        = 1.0
temperature_K = 300.0
converge      = true

[observe]
track = ["contact", "residence", "embedding", "diffusion_axis", "energy", "
        identity_entropy"]

[export]
write_xyz      = true
write_xyzf     = true
write_analysis_json = true
write_events_json = true
write_symbolic_trace_json = true
write_report_md = true
write_manifest_json = true

[outputs.identity_entropy]
enabled        = true
path           = "output/run.ie"
format         = "table"
precision      = 6
attach_to_viewer = true
write_per_frame = true
write_summary  = true

```

§ 10 C++ Implementation Skeleton

Proposed header and translation unit for the identity entropy output layer. C++17/20, no external dependencies beyond the standard library. Namespace: `vsim::ie`.

```

// vsim/identity_entropy.hpp

// vsim/identity_entropy.hpp -- Formation-First VSIM, Identity Entropy Output Layer
#pragma once
#include <array>
#include <cmath>
#include <cstdlib>
#include <fstream>
#include <stdexcept>
#include <string>
#include <vector>

namespace vsim::ie {

// --- Formation Context -----

struct FormationContext {
    std::string method;           // "deposition" | "cast" | "declared"
    std::string history;         // "tracked" | "partial" | "unknown"
    double      formation_time = 0.0;
};

```



```

    double    temperature_K    = 300.0;
    double    pressure_Pa      = 101325.0;
    std::string phase;          // "solid" | "liquid" | "gas" | "plasma"
    std::string defect_state;    // "unknown" | "tracked" | "seeded"
    std::string grain_state;     // "unknown" | "columnar" | "equiaxed"
    double    density           = 1.0;

    // M_f : fraction of formation history still recoverable.
    [[nodiscard]] double formation_memory() const noexcept;
};

// --- Identity Entropy Record -----
// One row in the .ie sidecar. All fields DERIVED -- never written back to state.

struct IERecord {
    uint64_t frame              = 0;
    uint64_t identity_id        = 0;

    double    J                 = 1.0; // Recoverable information /
        distinguishability
    double    dataloss           = 0.0; // 1 - J
    double    dataentropy        = 0.0; // Spread/ambiguity of identity channels
    double    residual           = 0.0; // Packing or projection residual magnitude
    double    hidden_activity     = 0.0; // Activity in non-projected channels
    double    projection_loss     = 0.0; // Loss: identity-space -> observable state
    double    recoverability      = 1.0; // 1 - projection_loss
    double    coalescence_loss    = 0.0; // Loss from higher-scale identity formation
    uint32_t lineage_depth       = 0; // Formation / coalescence layers behind
        object
    uint32_t state_quota         = 1; // Q_n : active identities at this scale
    double    entropy_flux       = 0.0; // d(identity entropy)/dt
    double    formation_memory    = 1.0; // Recoverable formation fraction (M_f)
    uint32_t flags               = 0;

    void sync() noexcept { // Enforce derived invariants before writing.
        dataloss           = 1.0 - J;
        recoverability      = 1.0 - projection_loss;
    }
};

// --- Coalescence Gate -----
// G_coal(n) = 1[m >= 3] * 1[d_bar <= l_bond] * 1[B_n >= tau_B]

struct CoalescenceInput {
    uint32_t member_count       = 0;
    double    mean_distance      = 0.0;
    double    bond_length        = 0.0;
    double    binding_score      = 0.0;
    double    tau_B              = 0.0;
};

[[nodiscard]] inline bool
coalescence_gate(const CoalescenceInput& in) noexcept {
    return (in.member_count >= 3)
        && (in.mean_distance <= in.bond_length)
        && (in.binding_score >= in.tau_B);
}

// R_coal(n) = || C_n - A_n( I*(n+1) ) ||
[[nodiscard]] double coalescence_residual(
    const std::vector<double>& cluster_state,
    const std::vector<double>& projected_identity) noexcept;

```

```
// --- .ie File Writer -----

class IEWriter {
public:
    explicit IEWriter(const std::string& path, int precision = 6);
    ~IEWriter();
    IEWriter(const IEWriter&) = delete;
    IEWriter& operator=(const IEWriter&) = delete;

    void write_header();
    void write(IERecord& rec); // calls sync() internally
    void write_summary();
    void close();

private:
    std::ofstream stream_;
    int precision_;
    uint64_t rows_written_ = 0;
    std::vector<IERecord> buffer_;
    bool header_written_ = false;
    bool closed_ = false;
};

} // namespace vsim::ie
```

```
// vsim/identity_entropy.cpp

// vsim/identity_entropy.cpp
#include "identity_entropy.hpp"
#include <algorithm>
#include <iomanip>
#include <numeric>

namespace vsim::ie {

double FormationContext::formation_memory() const noexcept {
    using B = bool;
    const std::array<B, 6> known = {{
        !phase.empty() && phase != "unknown",
        !defect_state.empty() && defect_state != "unknown",
        !grain_state.empty() && grain_state != "unknown",
        temperature_K > 0.0,
        pressure_Pa > 0.0,
        density > 0.0,
    }};
    const double n = static_cast<double>(std::count(known.begin(), known.end(),
        true));
    return n / static_cast<double>(known.size());
}

double coalescence_residual(
    const std::vector<double>& cluster_state,
    const std::vector<double>& projected_identity) noexcept
{
    if (cluster_state.size() != projected_identity.size()) return -1.0;
    double sq = 0.0;
    for (std::size_t i = 0; i < cluster_state.size(); ++i) {
        const double d = cluster_state[i] - projected_identity[i];
        sq += d * d;
    }
}
```

```

    return std::sqrt(sq);
}

IEWriter::IEWriter(const std::string& path, int precision)
    : precision_(precision)
{
    stream_.open(path, std::ios::out | std::ios::trunc);
    if (!stream_.is_open())
        throw std::runtime_error("IEWriter: cannot open " + path);
    stream_ << std::fixed << std::setprecision(precision_);
}

IEWriter::~IEWriter() { if (!closed_) close(); }

void IEWriter::write_header() {
    stream_ <<
        "frame identity_id J dataloss dataentropy residual hidden_activity "
        "projection_loss recoverability coalescence_loss "
        "lineage_depth state_quota entropy_flux formation_memory flags\n";
    header_written_ = true;
}

void IEWriter::write(IERecord& rec) {
    if (!header_written_) write_header();
    rec.sync();
    stream_
        << rec.frame << ' ' << rec.identity_id << ' '
        << rec.J << ' ' << rec.dataloss << ' ' << rec.dataentropy << ' '
        << rec.residual << ' ' << rec.hidden_activity << ' '
        << rec.projection_loss << ' ' << rec.recoverability << ' '
        << rec.coalescence_loss << ' '
        << rec.lineage_depth << ' ' << rec.state_quota << ' '
        << rec.entropy_flux << ' ' << rec.formation_memory << ' '
        << rec.flags << '\n';
    ++rows_written_;
    buffer_.push_back(rec);
}

void IEWriter::write_summary() {
    if (buffer_.empty()) return;
    const double mean_J =
        std::accumulate(buffer_.begin(), buffer_.end(), 0.0,
            [](double a, const IERecord& r){ return a + r.J; })
        / static_cast<double>(buffer_.size());
    const double min_J =
        std::min_element(buffer_.begin(), buffer_.end(),
            [](const IERecord& a, const IERecord& b){ return a.J < b.J; })->J;
    const auto coal =
        std::count_if(buffer_.begin(), buffer_.end(),
            [](const IERecord& r){ return r.coalescence_loss > 0.0; });
    stream_ << "\n# SUMMARY\n"
        << "# rows " << rows_written_ << '\n'
        << "# mean_J " << mean_J << '\n'
        << "# min_J " << min_J << '\n'
        << "# coal_events " << coal << '\n';
}

void IEWriter::close() {
    if (closed_) return;
    write_summary();
    stream_.close();
    closed_ = true;
}

```

```
}  
  
} // namespace vsim::ie
```

VSIM Internal · Theoretical–Simulation Bridge · v5.2.0-internal · Not for external distribution